```
import os
import sys
import numpy as np
import librosa
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout,Activation,Flatten
from tensorflow.keras.optimizers import Adam
from sklearn import metrics
import tensorflow.keras
import tensorflow.keras.layers as layers
import IPython.display as ipd
from tensorflow.keras.callbacks import ReduceLROnPlateau
import matplotlib.pyplot as plt
import pyswarms as ps
from tensorflow.keras.callbacks import EarlyStopping
from pyswarms.utils.search import RandomSearch
from pyswarms.single.global_best import GlobalBestPSO



print("Python version:", sys.version)
print("NumPy version:", np.__version__)
print("librosa version:", librosa.__version__)
print("TensorFlow version:", tf.__version__)
print("Pyswarms version:", ps.__version__)
```

```
Python version: 3.8.20 (default, Oct  3 2024, 15:19:54) [MSC v.1929 64 bit (AMD64)]
NumPy version: 1.19.5
librosa version: 0.9.2
TensorFlow version: 2.5.3
Pyswarms version: 1.3.0
```

```
def preprocess_file(song_path, n_mfcc=128, num_segments=14):
    """
    Preprocess a single song into MFCC feature segments.
    Each segment is reshaped to (16, 8, 1).
    """
    mfcc_segments = []

    try:
        duration = librosa.get_duration(filename=song_path)

        if duration > 90:  # longer than 1.5 minutes
            audio, sr = librosa.load(
                song_path,
                sr=None,
                offset=30,
                duration=60
            )
        else:
            audio, sr = librosa.load(song_path, sr=None)

        total_samples = len(audio)
        segment_length = total_samples // num_segments

        for s in range(num_segments):
            start = s * segment_length
            end = start + segment_length
            segment = audio[start:end]

            if len(segment) < segment_length // 2:
                continue

            mfcc = librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=n_mfcc)
            mfcc_mean = np.mean(mfcc.T, axis=0)
            mfcc_reshaped = np.reshape(mfcc_mean, (16, 8, 1))
            mfcc_segments.append(mfcc_reshaped)

    except Exception as e:
        print(f"Error processing {song_path}: {e}")

    return mfcc_segments


def preprocess_dataset(parent_dir, genres, n_mfcc=128, num_segments=14, max_files=None):
```

```
    dataset = []
    labels = []

    for genre, genre_number in genres.items():
        genre_dir = os.path.join(parent_dir, genre)
        print(f"Processing genre: {genre}")

        files = [f for f in os.listdir(genre_dir) if f.endswith((".wav", ".au", ".mp3"))]
        if max_files:
            files = files[:max_files]

        for filename in files:
            song_path = os.path.join(genre_dir, filename)
            mfcc_segments = preprocess_file(song_path, n_mfcc, num_segments)

            dataset.extend(mfcc_segments)
            labels.extend([genre_number] * len(mfcc_segments))

    dataset = np.array(dataset)
    labels = np.array(labels)

    print("Final dataset shape:", dataset.shape)
    print("Final labels shape:", labels.shape)
    return dataset, labels
```

```
genres = {
    'blues': 0, 'classical': 1, 'country': 2, 'disco': 3, 'hiphop': 4,
    'jazz': 5, 'metal': 6, 'pop': 7, 'reggae': 8, 'rock': 9
}

X, y = preprocess_dataset("data/genresWAV", genres)
```

```
Processing genre: blues
Processing genre: classical
Processing genre: country
Processing genre: disco
Processing genre: hiphop
Processing genre: jazz
Processing genre: metal
Processing genre: pop
Processing genre: reggae
Processing genre: rock
Final dataset shape: (14000, 16, 8, 1)
Final labels shape: (14000,)
```

```
labelencoder = LabelEncoder()
y = to_categorical(labelencoder.fit_transform(y))


from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=0)
```

```


# Define search space:
# [learning_rate, filters, dense_units, dropout]
bounds = ([1e-4, 16, 128],    # min values
          [1e-2, 64, 512])    # max values

def create_model(lr, filters, dense_units):
    model = models.Sequential([
        layers.Conv2D(int(filters), (3,3), activation='relu', padding='valid',input_shape=(16,8,1)),
        layers.MaxPooling2D(2, padding='same'),
        layers.Flatten(),
        layers.Dense(int(dense_units), activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    opt = tf.keras.optimizers.Adam(learning_rate=lr)
    model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['acc'])
    return model

def objective_function(params):
    losses = []
    for lr, filters, dense_units in params:


        model = create_model(lr, filters, dense_units)

        early_stop = EarlyStopping(
            monitor="val_loss",   # watch validation accuracy
            patience=3,               # stop if no improvement after 3 epochs
            restore_best_weights=True # rollback to best model
```

```
        )

        history = model.fit(
            X_train, y_train,
            validation_data=(X_test, y_test),
            epochs=15, batch_size=128, verbose=1  # keep short for search
            ,callbacks=[early_stop]
        )

        val_acc = history.history['val_acc'][-1]
        losses.append(1 - val_acc)  # minimize (1 - accuracy)
    return np.array(losses)
```

```
options = {'c1': 0.5, 'c2': 0.3, 'w': 0.9}

optimizer = GlobalBestPSO(n_particles=10, dimensions=3, options=options,
                          bounds=bounds)

# Optimize
best_cost, best_pos = optimizer.optimize(objective_function, iters=10)

print("Best params found:")
print("Learning rate:", best_pos[0])
print("Filters:", int(best_pos[1]))
print("Dense units:", int(best_pos[2]))
```

```
77/77 [==============================] - 1s 14ms/step - loss: 1.2137 - acc: 0.5792 - val_loss: 1.0737 - val_acc: 0.6231
Epoch 3/15
77/77 [==============================] - 1s 13ms/step - loss: 0.9957 - acc: 0.6580 - val_loss: 1.0979 - val_acc: 0.6281
Epoch 4/15
77/77 [==============================] - 1s 14ms/step - loss: 0.8556 - acc: 0.7034 - val_loss: 0.9281 - val_acc: 0.6929
Epoch 5/15
77/77 [==============================] - 1s 13ms/step - loss: 0.7556 - acc: 0.7433 - val_loss: 0.8699 - val_acc: 0.7133
Epoch 6/15
77/77 [==============================] - 1s 12ms/step - loss: 0.6832 - acc: 0.7636 - val_loss: 0.8512 - val_acc: 0.7186
Epoch 7/15
77/77 [==============================] - 1s 13ms/step - loss: 0.6207 - acc: 0.7855 - val_loss: 0.7655 - val_acc: 0.7483
Epoch 8/15
77/77 [==============================] - 1s 12ms/step - loss: 0.5775 - acc: 0.7995 - val_loss: 0.7572 - val_acc: 0.7500
Epoch 9/15
77/77 [==============================] - 1s 12ms/step - loss: 0.5193 - acc: 0.8200 - val_loss: 0.6930 - val_acc: 0.7698
Epoch 10/15
77/77 [==============================] - 1s 13ms/step - loss: 0.4535 - acc: 0.8410 - val_loss: 0.8318 - val_acc: 0.7360
Epoch 11/15
77/77 [==============================] - 1s 13ms/step - loss: 0.4415 - acc: 0.8435 - val_loss: 0.7252 - val_acc: 0.7769
Epoch 12/15
77/77 [==============================] - 1s 12ms/step - loss: 0.4026 - acc: 0.8574 - val_loss: 0.7797 - val_acc: 0.7681
Epoch 1/15
77/77 [==============================] - 2s 16ms/step - loss: 1.8056 - acc: 0.4621 - val_loss: 1.2319 - val_acc: 0.5888
Epoch 2/15
77/77 [==============================] - 1s 14ms/step - loss: 1.0723 - acc: 0.6342 - val_loss: 1.0639 - val_acc: 0.6443
Epoch 3/15
77/77 [==============================] - 1s 14ms/step - loss: 0.9458 - acc: 0.6773 - val_loss: 0.9565 - val_acc: 0.6833
Epoch 4/15
77/77 [==============================] - 1s 14ms/step - loss: 0.7651 - acc: 0.7467 - val_loss: 0.8306 - val_acc: 0.7260
Epoch 5/15
77/77 [==============================] - 1s 13ms/step - loss: 0.6542 - acc: 0.7866 - val_loss: 0.7357 - val_acc: 0.7640
Epoch 6/15
77/77 [==============================] - 1s 14ms/step - loss: 0.5772 - acc: 0.8139 - val_loss: 0.7113 - val_acc: 0.7671
Epoch 7/15
77/77 [==============================] - 1s 15ms/step - loss: 0.5081 - acc: 0.8335 - val_loss: 0.7155 - val_acc: 0.7581
Epoch 8/15
77/77 [==============================] - 1s 14ms/step - loss: 0.4433 - acc: 0.8585 - val_loss: 0.5997 - val_acc: 0.8005
Epoch 9/15
77/77 [==============================] - 1s 14ms/step - loss: 0.3806 - acc: 0.8791 - val_loss: 0.6126 - val_acc: 0.8012
Epoch 10/15
77/77 [==============================] - 1s 13ms/step - loss: 0.3544 - acc: 0.8863 - val_loss: 0.5380 - val_acc: 0.8248
Epoch 11/15
77/77 [==============================] - 1s 14ms/step - loss: 0.2889 - acc: 0.9147 - val_loss: 0.5873 - val_acc: 0.8083
Epoch 12/15
77/77 [==============================] - 1s 15ms/step - loss: 0.2469 - acc: 0.9268 - val_loss: 0.5021 - val_acc: 0.8386
Epoch 13/15
77/77 [==============================] - 1s 14ms/step - loss: 0.2276 - acc: 0.9322 - val_loss: 0.4699 - val_acc: 0.8481
Epoch 14/15
77/77 [==============================] - 1s 13ms/step - loss: 0.1824 - acc: 0.9494 - val_loss: 0.4626 - val_acc: 0.8543
Epoch 15/15
77/77 [==============================] - 1s 13ms/step - loss: 0.1588 - acc: 0.9566 - val_loss: 0.4915 - val_acc: 0.8519
pyswarms.single.global_best: 100%|██████████████████████|10/10, best_cost=0.13
2026-06-27 00:26:28,082 - pyswarms.single.global_best - INFO - Optimization finished | best cost: 0.130476176738739, best
Best params found:
Learning rate: 0.0010486934124383427
Filters: 50
Dense units: 421
```